Problema del Vuelto del Cajero

Solución con Programación Dinámica (Bottom-Up)

Datos del Problema

Billetes disponibles: [20, 20, 50, 50, 50, 100] soles

Cantidad de billetes: n = 6

Dimensiones de DP: (n+1) × (vuelto+1), inicializada en F (falso)

Pseudocódigo

```
Algoritmo VueltoCajero
 Inicio
 Variables
        billetes: lista de los billetes disponibles
        n: cantidad de billetes
        vuelto: monto del vuelto requerido
        DP: matriz booleana de dimensiones (n+1) × (vuelto+1)
             DP[i][j] = verdadero si con los primeros i billetes
                        se puede formar exactamente la suma j

 función puedeEntregarVuelto(billetes, n, vuelto): booleano
 Inicio
    Crear matriz DP de tamaño (n+1) × (vuelto+1) inicializada en falso

    // Caso base: con cualquier cantidad de billetes se puede formar suma 0
    Para i desde 0 hasta n hacer:
        DP[i][0] = verdadero
    Fin Para

    // Llenado bottom-up
    Para i desde 1 hasta n hacer:
        Para j desde 1 hasta vuelto hacer:
            // Opción 1: No usar el billete i
            DP[i][j] = DP[i-1][j]

            // Opción 2: Usar el billete i (solo si cabe)
            Si billetes[i] <= j entonces
                Si DP[i-1][j - billetes[i]] es verdadero entonces
                    DP[i][j] = verdadero
                Fin Si
            Fin Si
```

```
        Fin Para
    Fin Para

    retornar DP[n][vuelto]
 Fin puedeEntregarVuelto

 Bloque Principal
    Leer n
    Leer billetes (de tamaño n, indexados desde 1)
    Leer vuelto

    puedeEntregarVuelto(billetes, n, vuelto)

    // Imprimir la matriz DP completa
    Mostrar "Matriz de soluciones parciales DP:"
    Mostrar encabezado de columnas: j = 0, 1, 2, ..., vuelto
    Para i desde 0 hasta n hacer:
        Mostrar fila i de DP
    Fin Para

    // Resultado
    Si DP[n][vuelto] es verdadero entonces
        Mostrar "Puede entregar el vuelto"

        // Recuperar cuáles billetes se usaron
        j = vuelto
        Para i desde n hasta 1 retrocediendo hacer:
            Si DP[i][j] ≠ DP[i-1][j] entonces
                Mostrar "Usar billete ", i, " de valor ", billetes[i]
                j = j - billetes[i]
            Fin Si
        Fin Para
    Sino
        Mostrar "No se puede entregar el vuelto"
    Fin Si
 Fin Bloque Principal
Fin Algoritmo VueltoCajero
```

Ejecución Paso a Paso — Caso 1: Vuelto = 270

Constante mágica: 270 soles

Paso 0: Caso base — columna j=0

Toda la columna j=0 se marca como V (con cualquier cantidad de billetes se puede formar suma 0, no entregando ninguno).

```
DP[0][0]=V, DP[1][0]=V, DP[2][0]=V, DP[3][0]=V, DP[4][0]=V, DP[5][0]=V, DP[6][0]=V
```

Paso 1: i=1, billete[1] = 20

Se recorre j desde 1 hasta 270. Para cada j se evalúan las dos opciones:

```
DP[1][j] = DP[0][j]                        → no usar billete 1
Si 20 ≤ j y DP[0][j-20] = V → DP[1][j] = V → usar billete 1

j=20: DP[0][20]=F, pero 20≤20 y DP[0][0]=V   → DP[1][20] = V ✓
j=40: DP[0][40]=F, 20≤40 pero DP[0][20]=F    → DP[1][40] = F
...todos los demás j: F
```

Fila i=1: DP[1][0]=V, DP[1][20]=V. Solo se puede formar 0 o 20.

Paso 2: i=2, billete[2] = 20

```
j=20: DP[1][20]=V                            → V ✓ (heredado, sin usar billete 2)
j=40: DP[1][40]=F, pero 20≤40 y DP[1][20]=V   → V ✓ (usar billete 2)
j=50: DP[1][50]=F, 20≤50 pero DP[1][30]=F     → F
...todos los demás j: F
```

Fila i=2: 0, 20, 40. Con dos billetes de 20 se forma 0, 20 o 40.

Paso 3: i=3, billete[3] = 50

```
j=20: DP[2][20]=V                            → V ✓ (heredado)
j=40: DP[2][40]=V                            → V ✓ (heredado)
j=50: DP[2][50]=F, pero 50≤50 y DP[2][0]=V    → V ✓ (usar billete 3)
j=70: DP[2][70]=F, pero 50≤70 y DP[2][20]=V   → V ✓ (20 + 50)
j=90: DP[2][90]=F, pero 50≤90 y DP[2][40]=V   → V ✓ (20 + 20 + 50)
j=100: DP[2][100]=F, 50≤100 pero DP[2][50]=F  → F
```

Fila i=3: 0, 20, 40, 50, 70, 90.

Paso 4: i=4, billete[4] = 50

```
Heredados de fila 3: 0, 20, 40, 50, 70, 90
Nuevos al sumar 50 a valores existentes:
  50+50=100  → DP[4][100] = V ✓ NUEVO
  70+50=120  → DP[4][120] = V ✓ NUEVO
  90+50=140  → DP[4][140] = V ✓ NUEVO
```

Fila i=4: 0, 20, 40, 50, 70, 90, 100, 120, 140.

Paso 5: i=5, billete[5] = 50

```
Heredados de fila 4: 0, 20, 40, 50, 70, 90, 100, 120, 140
Nuevos al sumar 50 a valores existentes:
  100+50=150 → DP[5][150] = V ✓ NUEVO
  120+50=170 → DP[5][170] = V ✓ NUEVO
  140+50=190 → DP[5][190] = V ✓ NUEVO
```

Fila i=5: 0, 20, 40, 50, 70, 90, 100, 120, 140, 150, 170, 190.

Paso 6: i=6, billete[6] = 100

```
Heredados de fila 5: 0, 20, 40, 50, 70, 90, 100, 120, 140, 150, 170, 190
Nuevos al sumar 100 a valores existentes:
  100+100=200 → DP[6][200] = V ✓ NUEVO
  120+100=220 → DP[6][220] = V ✓ NUEVO
  140+100=240 → DP[6][240] = V ✓ NUEVO
  150+100=250 → DP[6][250] = V ✓ NUEVO
  170+100=270 → DP[6][270] = V ✓ NUEVO
  190+100=290 → fuera de rango
```

Fila i=6: 0, 20, 40, 50, 70, 90, 100, 120, 140, 150, 170, 190, 200, 220, 240, 250, 270.

Matriz DP Completa — Caso 1 (vuelto = 270)

Se muestran solo las columnas donde al menos un valor es V. Las columnas intermedias son todas F.

	j=0	j=20	j=40	j=50	j=70	j=90	j=100	j=120	j=140	j=150	j=170	j=190	j=200	j=220	j=240	j=250	j=270
i=0	V	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F
i=1(20)	V	V	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F
i=2(20)	V	V	V	F	F	F	F	F	F	F	F	F	F	F	F	F	F
i=3(50)	V	V	V	V	V	V	F	F	F	F	F	F	F	F	F	F	F
i=4(50)	V	V	V	V	V	V	V	V	V	F	F	F	F	F	F	F	F
i=5(50)	V	V	V	V	V	V	V	V	V	V	V	V	F	F	F	F	F
i=6(100)	V	V	V	V	V	V	V	V	V	V	V	V	V	V	V	V	**V**

DP[6][270] = V → **Puede entregar el vuelto**

Recuperación de Billetes (recorrido inverso)

```
j = 270
i=6: DP[6][270]=V, DP[5][270]=F → Distintos → Usar billete 6 (100). j = 270-100 = 170
i=5: DP[5][170]=V, DP[4][170]=F → Distintos → Usar billete 5 (50).  j = 170-50  = 120
i=4: DP[4][120]=V, DP[3][120]=F → Distintos → Usar billete 4 (50).  j = 120-50  = 70
i=3: DP[3][70]=V,  DP[2][70]=F  → Distintos → Usar billete 3 (50).  j = 70-50   = 20
i=2: DP[2][20]=V,  DP[1][20]=V  → Iguales   → No usar billete 2
i=1: DP[1][20]=V,  DP[0][20]=F  → Distintos → Usar billete 1 (20).  j = 20-20   = 0
```

Billetes usados: 20 + 50 + 50 + 50 + 100 = 270 ✓

Ejecución — Caso 2: Vuelto = 230

Revisando la fila i=6 de la matriz DP, las sumas alcanzables con los billetes {20, 20, 50, 50, 50, 100} son:

```
0, 20, 40, 50, 70, 90, 100, 120, 140, 150, 170, 190, 200, 220, 240, 250, 270
```

El valor 230 no aparece en esta lista.

```
DP[6][230] = F → No se puede entregar el vuelto
```

No existe ninguna combinación de los billetes disponibles que sume exactamente 230 soles. El cajero no podrá entregar este vuelto con los billetes que tiene.